10

Apache Spark từ số 0

DataFrame · lazy evaluation · shuffle

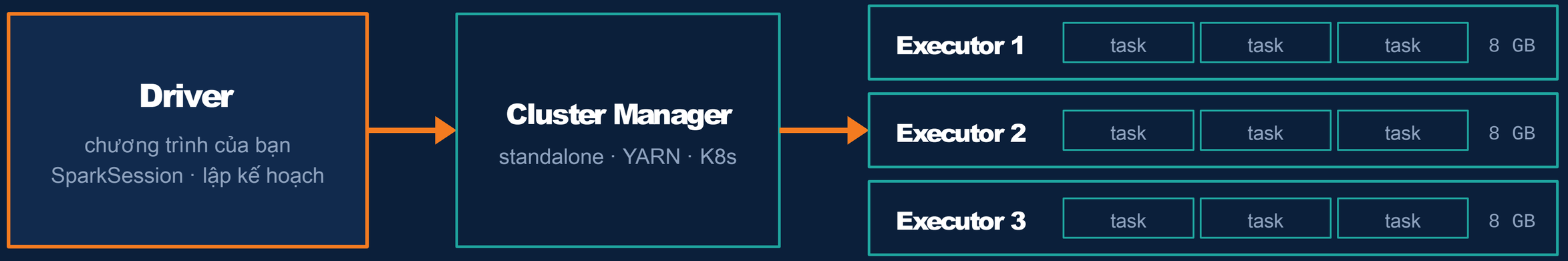
Hôm nay mình nói gì

1. Khi nào cần (và không cần) Spark
2. Driver, executor, cluster manager
3. Transformation vs action · DAG & lazy evaluation
4. Shuffle · Partition, cache
5. Đọc Spark UI

Khi nào KHÔNG cần Spark

- Dữ liệu < 50 GB → DuckDB / Polars nhanh hơn, rẻ hơn
- Một máy 64 GB RAM làm được rất nhiều
- Spark trả giá bằng độ phức tạp, khởi động chậm
- Cần Spark khi không vừa một máy, hoặc cần streaming

Kiến trúc Spark



Driver lập kế hoạch, executor làm việc.

Job PySpark đầu tiên

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("daily_revenue").getOrCreate()

df = spark.read.parquet("s3a://lake/silver/pos/pos_clean/")

revenue = (
    df.filter(F.col("dt") >= "2026-11-01")
      .groupBy("store_id", "dt")
      .agg(F.sum("amount").alias("revenue"))
)

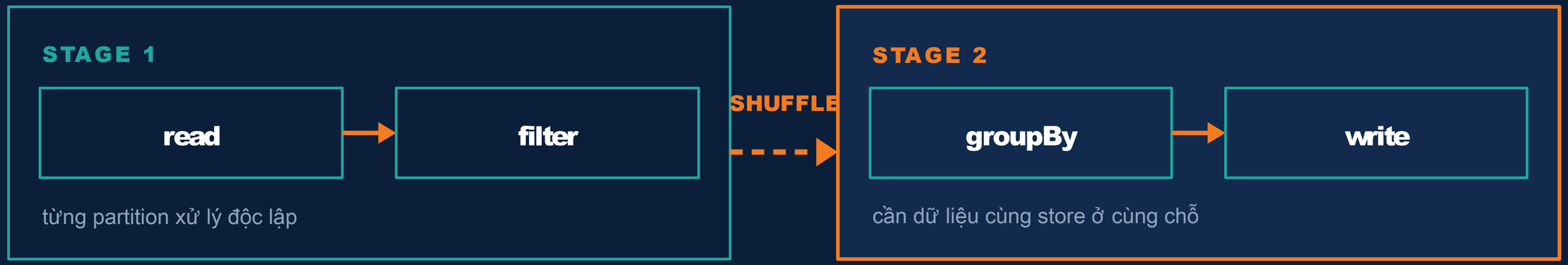
revenue.write.mode("overwrite").parquet("s3a://lake/gold/sales/daily_revenue/")
```

Chưa có gì chạy cho đến dòng cuối.

Transformation vs Action

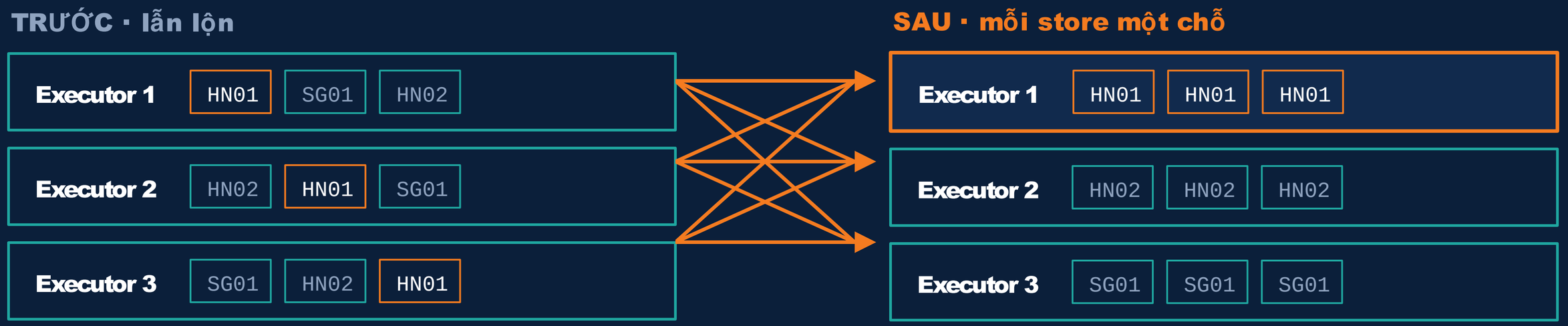
- **Transformation chỉ ghi kế hoạch:** `filter select join groupBy`
- **Action mới chạy:** `count collect write show`
- **Lazy = Spark tối ưu toàn bộ kế hoạch trước khi chạy**
- `collect()` **kéo hết về driver → nguy hiểm**

DAG của job



Mỗi lần dữ liệu phải đổi chỗ giữa executor là một stage mới.

Shuffle là gì



Shuffle = ghi đĩa + truyền mạng + đọc lại. Đắt nhất trong Spark.

Thao tác nào gây shuffle

- **Shuffle:** `groupBy join distinct repartition orderBy`
- **Không shuffle:** `filter select map union`
- **Giảm số shuffle = giảm thời gian job**

Partition trong Spark

- Đơn vị song song
- Mặc định shuffle ra 200: `spark.sql.shuffle.partitions`
- Quá ít → không tận dụng lõi · quá nhiều → overhead
- `coalesce` trước khi ghi để tránh small file

Cache và repartition trước khi ghi

```
clean = spark.read.parquet("s3a://lake/silver/pos/pos_clean/").filter("dt >= '2026-11-01'")
clean.cache()                          # dùng 2 lần bên dưới

by_store = clean.groupBy("store_id", "dt").agg(F.sum("amount").alias("revenue"))
by_product = clean.groupBy("product_id", "dt").agg(F.sum("quantity").alias("qty"))

(by_store.repartition("dt")
         .write.mode("overwrite").partitionBy("dt")
         .parquet("s3a://lake/gold/sales/daily_revenue/"))

clean.unpersist()
```

Cache chỉ khi dùng lại, và nhớ unpersist.

Đọc Spark UI

Jobs | Stages | Storage | SQL — localhost:4040

Stage	Description	Tasks	Duration	Shuffle Read	Shuffle Write
0	parquet scan + filter	48/48	[SỐ] s	—	[SỐ] MB
1	groupBy store_id, dt	200/200	[SỐ] s	[SỐ] MB	—
2	write parquet	24/24	[SỐ] s	—	—

Stage nào lâu nhất, shuffle bao nhiêu — bắt đầu từ đó.

Lỗi hay gặp

SAI	ĐÚNG
`df.collect()` ~~rồi xử lý bằng Python~~	Giữ trên DataFrame
`df.count()` ~~khắp nơi để debug~~	Mỗi count là một job
~~Để 200 shuffle partition cho mọi job~~	Chỉnh theo dữ liệu
~~Spark cho 500 MB dữ liệu~~	DuckDB

Lab #10 — Xử lý 10 triệu dòng, xem Spark UI

1. `make up PROFILE=all`, mở Jupyter PySpark
2. Đọc silver/pos (10 triệu dòng), doanh thu theo store, ngày
3. Mở Spark UI `localhost:4040`, đếm stage và shuffle size
4. Đổi shuffle.partitions 200 → 24, so thời gian
5. Ghi kết quả vào gold

KẾT QUẢ MONG ĐỢI

Thấy **2 stage**

Thời gian giảm sau khi chỉnh partition

CÂU HỎI PHỎNG VẤN #10

Shuffle là gì và vì sao đắt?

Dữ liệu đổi chỗ giữa executor: đĩa, mạng, serialize.

VIDEO TIẾP THEO

11

Tối ưu Spark: skew, broadcast join, AQE, đọc Spark UI

Thứ Ba 8/12 · 19:00

Đăng ký